Classification: Naïve Bayes' Classifier

Dr. Nikhilanand Arya

IIITA

Naïve Bayesian Classifier

Overview

- A statistical classifier
- Performs *probabilistic prediction*
- Predicts class membership probabilities

Key Assumptions

1. Classes are **mutually exclusive & exhaustive**
2. Attributes are **independent given class**

Foundation

- Based on **Bayes' Theorem**

Why "Naïve"?

- Strong independence assumption
- Rarely true in real data
- Still works surprisingly well!

Advantages

- Empirically effective
- Scales very well

Naïve Bayes Classifier Example 1: Training Dataset

Class

- play golf = **yes**
- play golf = **no**

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

Test Instance

Outlook	Temp	Humidity	Windy	Play
Rainy	Cool	High	True	?

Bayesian Classification: Motivation

Nature of Classification

- Relationship between attributes and class is **non-deterministic**
- Exact classification is often **not possible**

Probabilistic Approach

- Classification is done **probabilistically**
- Assigns likelihood to each class

Bayesian Classifier

- Models **probabilistic relationships** between attributes and class labels

Core Principle

- Based on **Bayes' Theorem of Probability**

Next Step

- Theory of Probability
- Bayes' Theorem

Simple Probability

- Suppose A and B are two events and $P(A), P(B)$ denote their probabilities

Mutually Exclusive Events

Two events are **mutually exclusive** if the occurrence of one prevents the occurrence of the other

Examples

- Tossing a coin (two events)
- Tossing a dice (six events)

Think!

Can you give an example where two events are **not mutually exclusive**?

Hint: Tossing two coins, Weather (sunny, foggy, warm)

Mutually Exclusive vs Independent Events

Mutually Exclusive

- Cannot occur together
- If A occurs, B cannot occur
- $P(A \cap B) = 0$

Example:

- Head or Tail (single coin toss)
- Dice outcome: 2 or 5

Independent Events

- Occurrence of one does not affect the other
- Events are unrelated
- $P(A \cap B) = P(A)\,P(B)$

Example:

- Coin toss and dice roll
- Weather and exam score (approx.)

Key Insight

Mutually exclusive events **cannot be independent** (except trivial case when probability is zero)

Joint Probability

Definition

If $P(A)$ and $P(B)$ are probabilities of two events:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Mutually Exclusive

$$P(A \cap B) = 0$$

$$\Rightarrow \; P(A \cup B) = P(A) + P(B)$$

Independent Events

$$P(A \cap B) = P(A) \cdot P(B)$$

Key Idea

- Union depends on overlap
- Intersection captures joint occurrence
- Special cases simplify computation

Conditional Probability

Definition

If events are dependent, their probability is expressed using **conditional probability**

- Probability of A given B is denoted as $P(A|B)$
- Assumes $P(B) \neq 0$

Interpretation

- Restrict sample space to event B
- Compute probability of A within B

Formula

$$P(A|B) = \frac{\text{Number of favorable outcomes in } A \cap B}{\text{Number of outcomes in } B}$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Conditional Probability: Generalization

Generalization

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B \cap A)}{P(B)}$$

Using Product Rule

$$P(A \cap B) = P(B|A) \cdot P(A)$$

$$P(A \cap B) = P(A|B) \cdot P(B)$$

Using Total Probability

$$P(B) = P(B|A) \cdot P(A) + P(B|\bar{A}) \cdot P(\bar{A})$$

, where $\bar{A}$ denotes the compliment of event A.

Final Expression

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B|A) \cdot P(A) + P(B|\bar{A}) \cdot P(\bar{A})}$$

Total Probability

Definition

Let $E_1, E_2, \ldots, E_n$ be **mutually exclusive** and **exhaustive** events

- They form a partition of the sample space
- Exactly one event occurs at a time

Formula

$$P(A) = \sum_{i=1}^{n} P(E_i)\, P(A|E_i)$$

Interpretation

Break a complex event into simpler conditional cases

Expanded Form

$$P(A) = P(E_1)P(A|E_1) + P(E_2)P(A|E_2) + \cdots + P(E_n)P(A|E_n)$$

Bayes' Theorem

Theorem

Let $E_1, E_2, \ldots, E_n$ be **mutually exclusive** and **exhaustive** events

- They form a partition of the sample space
- Event A occurs under one of these cases

Interpretation

Update belief about E_i after observing A

Bayes' Formula

$$P(E_i|A) = \frac{P(E_i)\,P(A|E_i)}{\sum_{i=1}^{n} P(E_i)\,P(A|E_i)}$$

Key Idea

- Posterior = Prior $\times$ Likelihood
- Normalize using total probability

Bayes' Theorem

- Bayes' Theorem:

$$P(H|X) = \frac{P(X|H)\,P(H)}{P(X)} \propto P(X|H)\,P(H)$$

Posterior Probability
What we should choose

Likelihood
What we just see

Prior Probability
What we knew previously

- X: a data sample (*evidence*)
- H: X belongs to class C

Prediction

Prediction can be done based on Bayes' Theorem

Classification Rule

Classification is to derive the **maximum posterior**

Prior and Posterior Probabilities

X	x_1	x_2	x_3	x_3	x_2	x_1	x_1	x_3	x_2	x_2
Y	A	A	B	A	B	A	B	B	B	A

Concept

- $P(A), P(B)$ are **prior probabilities**
- $P(A|B), P(B|A)$ are **posterior probabilities**

Example

Event Y has outcomes A, B depending on $X = \{x_1, x_2, x_3\}$

- **Case 1 (Prior):**

$$P(Y = A) = \frac{5}{10} = 0.5$$

- **Case 2 (Posterior):**

$$P(X = x_2 \mid Y = A) = \frac{2}{5} = 0.4$$

Conclusion

Posterior = Conditional probability
Prior = Initial probability

Naïve Bayes Classifier: Derivation

Bayes' Theorem

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

- $X = (x_1, x_2, \ldots, x_n)$ is the feature vector
- C is the class label

Simplification

Since $P(X)$ is constant:

$$P(C \mid X) \propto P(X \mid C)\, P(C)$$

Computing Likelihood

Likelihood Term

$$P(X \mid C) = P(x_1, x_2, \ldots, x_n \mid C)$$

Using chain rule:

$$P(X|C) = P(x_1|C) \cdot P(x_2|x_1, C) \cdots P(x_n|x_1, \ldots, C)$$

Problem

- Requires modeling complex dependencies
- Needs huge amount of data

Naïve Assumption

Key Assumption

Features are **conditionally independent** given the class

$$P(x_i \mid x_1, x_2, \ldots, C) = P(x_i \mid C)$$

Result

$$P(X \mid C) \approx \prod_{i=1}^{n} P(x_i \mid C)$$

Naïve Bayes Model

Final Model

$$P(C \mid X) \propto P(C) \prod_{i=1}^{n} P(x_i \mid C)$$

- Prior: $P(C)$
- Likelihood: $P(x_i \mid C)$

Classification Rule

$$\hat{C} = \arg\max_{C} \; P(C) \prod_{i=1}^{n} P(x_i \mid C)$$

Why Naïve Bayes Works

- Reduces complexity: exponential $\rightarrow$ linear
- Easy to compute using counts
- Requires less training data

Key Insight

Each feature contributes **independent evidence**

Surprising Result

Despite strong assumptions, performance is often very good

Naïve Bayes Classifier Example 1: Training Dataset

Class

- play golf = **yes**
- play golf = **no**

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

Test Instance

Outlook	Temp	Humidity	Windy	Play
Rainy	Cool	High	True	?

Naïve Bayes Classifier Example: $P(\text{Yes} \mid \text{Sunny})$

Frequency Table		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3

$$P(x \mid c) = P(Sunny \mid Yes) = 3/9 = 0.33$$

Likelihood Table		Play Golf		
		Yes	No	
Outlook	Sunny	3/9	2/5	5/14
	Overcast	4/9	0/5	4/14
	Rainy	2/9	3/5	5/14
		9/14	5/14	

$$P(x) = P(Sunny) = 5/14 = 0.36$$

$$P(c) = P(Yes) = 9/14 = 0.64$$

Posterior Probability: $P(c \mid x) = P(Yes \mid Sunny) = 0.33 \times 0.64 \div 0.36 = 0.60$

Naïve Bayes Classifier Example: $P(\text{No} \mid \text{Sunny})$

Frequency Table		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
		9	5	14

$$P(x \mid c) = P(Sunny \mid No) = 2/5 = 0.4$$

$$P(x) = P(Sunny) = 5/14 = 0.36$$

$$P(c) = P(No) = 5/14 = 0.36$$

Posterior Probability: $P(c \mid x) = P(No \mid Sunny) = 0.40 \times 0.36 \div 0.36 = 0.40$

Naïve Bayes Classifier Example: Likelihood Tables

Frequency Table

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3

Likelihood Table

		Play Golf	
		Yes	No
Outlook	Sunny	3/9	2/5
	Overcast	4/9	0/5
	Rainy	2/9	3/5

		Play Golf	
		Yes	No
Humidity	High	3/9	4/5
	Normal	6/9	1/5

		Play Golf	
		Yes	No
Temp.	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5

		Play Golf	
		Yes	No
Windy	False	6/9	2/5
	True	3/9	3/5

Naïve Bayes Classifier Example: Classification

Outlook	Temp	Humidity	Windy	Play
Rainy	Cool	High	True	?

$$P(Yes \mid X) = P(Rainy \mid Yes) \times P(Cool \mid Yes) \times P(High \mid Yes) \times P(True \mid Yes) \times P(Yes) \;/\; P(x)$$

$$P(Yes \mid X) = \frac{2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14}{P(x)} = 0.00529 \rightarrow 0.2 = \frac{0.00529}{0.02057 + 0.00529}$$

$$P(No \mid X) = P(Rainy \mid No) \times P(Cool \mid No) \times P(High \mid No) \times P(True \mid No) \times P(No) \;/\; P(x)$$

$$P(No \mid X) = \frac{3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14}{P(x)} = 0.02057 \rightarrow 0.8 = \frac{0.02057}{0.02057 + 0.00529}$$

$P(x) = P(x \mid yes) * P(yes) + P(x \mid no) * P(no)$

Algorithm: Naïve Bayes Classifier

1. Compute prior probabilities
2. Compute likelihoods
3. Multiply probabilities
4. Choose maximum posterior

Avoiding the Zero-Probability Problem

Problem

Naïve Bayes requires all conditional probabilities to be **non-zero**

Issue

If any probability is zero:

$$P(C \mid X) = P(C) \prod_i P(x_i \mid C) = 0$$

- One zero term makes the entire prediction zero
- Leads to incorrect classification

Example of Zero Probability

Dataset

Total samples = 1000

- income = low: 0
- income = medium: 990
- income = high: 10

Without Correction

$$P(\text{income} = \text{low}) = \frac{0}{1000} = 0$$

- Any test sample with **income = low** $\rightarrow$ probability becomes 0
- Model fails completely

Laplacian Correction (or Laplacian estimator)

Solution

Add 1 or or a small integer to each count (Laplace smoothing)

Formula

$$P(x_i \mid C) = \frac{\text{count}(x_i) + 1}{N + k}$$

- N = total samples
- k = number of possible values

Corrected Probabilities

Adjusted Counts

$$N = 1000, \quad k = 3 \Rightarrow N + k = 1003$$

- $P(\text{low}) = \frac{1}{1003}$
- $P(\text{medium}) = \frac{991}{1003}$
- $P(\text{high}) = \frac{11}{1003}$

Observation

No probability is zero anymore

Intuition Behind Smoothing

Key Idea

We assume unseen events are **rare, not impossible**

- Training data may be incomplete
- Add small probability for unseen cases

Interpretation

"Never seen" $\neq$ "Impossible"

Summary

- Zero probability breaks Naïve Bayes
- Laplace smoothing prevents zero values
- Keeps probabilities realistic

Important

Corrected probabilities remain close to original values

Result

More robust and reliable classification

Naïve Bayes: Strengths and Weaknesses

Strengths

1. **Performance**
   - Comparable to Decision Trees and Neural Networks
   - Works well even with small datasets
2. **Incremental Learning**
   - Each new example updates probabilities
   - Combines prior knowledge with observed data

Weaknesses

Independence Assumption

- Assumes attributes are conditionally independent
- Often unrealistic in real-world data

Example:

- Profile: (age, family history)
- Symptoms: (fever, cough)
- Disease: (lung cancer, diabetes)

Dependencies among these cannot be modeled

Handling Different Types of Features

Key Idea

Naïve Bayes handles features differently based on their type:

- **Categorical features** $\rightarrow$ Frequency-based probability
- **Continuous features** $\rightarrow$ Gaussian (Normal) distribution

Categorical Features

Formula

$$P(x_k = v_k \mid C_i) = \frac{\#\text{ tuples in } C_i \text{ with } x_k = v_k}{|C_i|}$$

Example

Class: Play Golf = Yes

Feature: Outlook

- Sunny appears 3 times out of 9

$$P(\text{Sunny} \mid \text{Yes}) = \frac{3}{9} = 0.33$$

Intuition: Categorical Case

- Count how often a value occurs within a class
- Divide by total examples of that class

Key Insight

It is simply a **relative frequency**

Interpretation

Among all "Yes" cases, how often is Outlook = Sunny?

Continuous Features

Problem

Continuous values cannot be counted directly

Solution

Assume data follows a **Gaussian (Normal) distribution**

Formula

$$P(x_k = v_k \mid C_i) = \frac{1}{\sqrt{2\pi}\,\sigma_{C_i}} \exp\left(-\frac{(x-\mu_{C_i})^2}{2\sigma_{C_i}^2}\right)$$

Example: Continuous Feature

Example

Feature: Age

Class: Disease = Yes

- Mean (μ) = 50
- Standard deviation (σ) = 10

- For a patient with age = 60:

$$P(\text{Age} = 60 \mid \text{Yes}) = \frac{1}{\sqrt{2\pi} \cdot 10} e^{-\frac{(60-50)^2}{2 \cdot 10^2}}$$

Intuition: Continuous Case

- Instead of counting $\rightarrow$ estimate probability using a curve
- Values near mean $\rightarrow$ higher probability
- Values far from mean $\rightarrow$ lower probability

Key Insight

Gaussian distribution models how values are spread

Interpretation

"How likely is this value given the class distribution?"

Naive Bayes Code Structure

```
import numpy as np

class NaiveBayes:
    def __init__(self):
        self.classes = None
        self.mean = None
        self.var = None
        self.priors = None
```

Initialization in Training

```
def fit(self, X, y):
    n_samples, n_features = X.shape
    self.classes = np.unique(y)

    n_classes = len(self.classes)

    self.mean = np.zeros((n_classes, n_features))
    self.var = np.zeros((n_classes, n_features))
    self.priors = np.zeros(n_classes)
```

Goal

Estimate class-wise statistics

Training: Parameter Estimation

```
for idx, c in enumerate(self.classes):
    X_c = X[y == c]

    self.mean[idx, :] = X_c.mean(axis=0)
    self.var[idx, :] = X_c.var(axis=0)

    self.priors[idx] = X_c.shape[0] / float(n_samples)
```

Interpretation

Learn distribution for each class

Gaussian Likelihood

```
def _pdf(self, class_idx, x):
    mean = self.mean[class_idx]
    var = self.var[class_idx]

    numerator = np.exp(- (x - mean)**2 / (2 * var))
    denominator = np.sqrt(2 * np.pi * var)

    return numerator / denominator
```

Key Idea

Assume features follow Gaussian distribution

Posterior Computation

```
def _predict(self, x):
    posteriors = []

    for idx, c in enumerate(self.classes):
        prior = np.log(self.priors[idx])
        likelihood = np.sum(np.log(self._pdf(idx, x)))

        posterior = prior + likelihood
        posteriors.append(posterior)

    return self.classes[np.argmax(posteriors)]
```

Why log?

Avoid numerical underflow

Prediction

```
def predict(self, X):
    y_pred = [self._predict(x) for x in X]
    return np.array(y_pred)
```

Output

Returns predicted class labels

Naive Bayes Intuition

$$P(y|x) \propto P(y) \prod_i P(x_i|y)$$

- Assumes feature independence
- Fast and simple
- Works well for high-dimensional data

Core Idea

Choose class with highest posterior probability

Reference

- Data Mining: Concepts and Techniques – Jiawei Han
- Introduction to Data Mining – Tan et al.

Thank You

Any Questions?